

RNN (Recurrent Neural Network)

Fall 2025

School of IT Convergence

Prof. Dongsup Jin

RNN (1)

- RNN (Recurrent Neural Network)

- A neural network

- Train sequential data or time series data
 - Describe an array of indeterminate length

- (Ex)

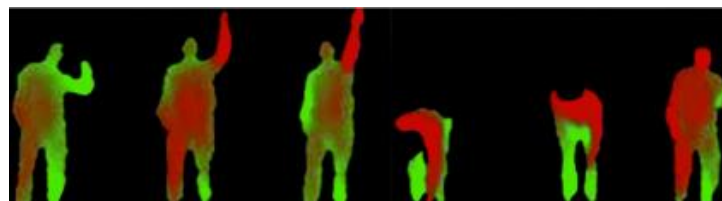
- Text data

I live to eat (vs) I eat to live

- Financial data (stock or coin)

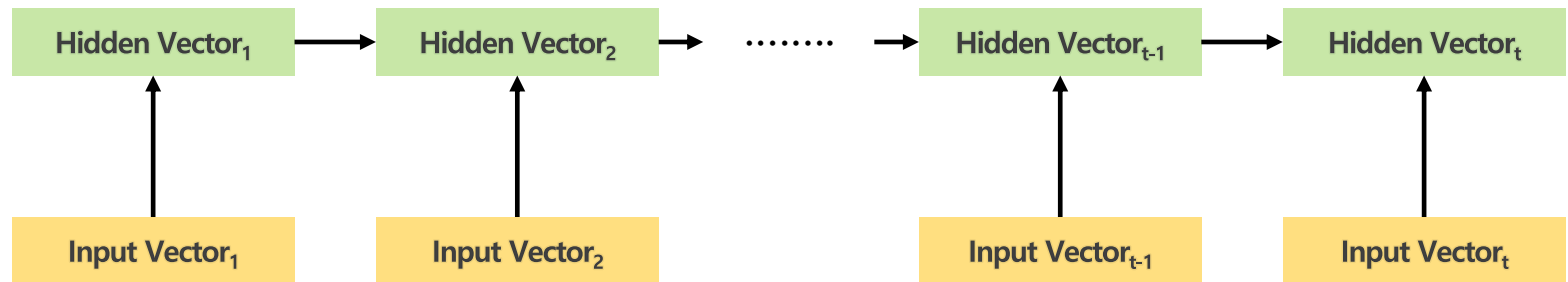


- Action or Gesture data



RNN (2)

- RNN (Recurrent Neural Network)
 - Create a hidden vector by synthesizing the sequential input vectors up to the current time (t)
 - Create the current hidden vector from the previous hidden vector (t-1) and current input vector (t)
 - Representative RNNs: [LSTM \(Long Short-Term Memory\)](#), [GRU \(Gated Recurrent Unit\)](#)

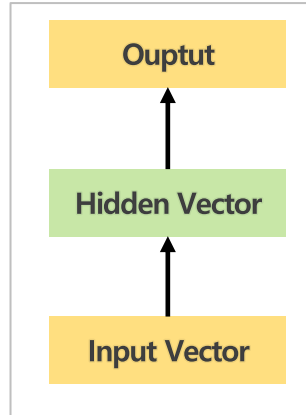


RNN structure

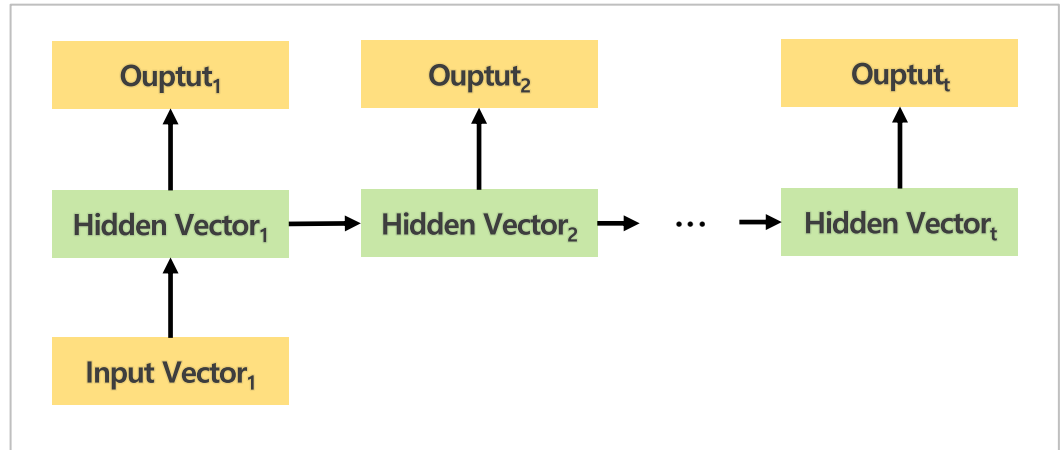
RNN (3)

- Typical RNN Structures (1)

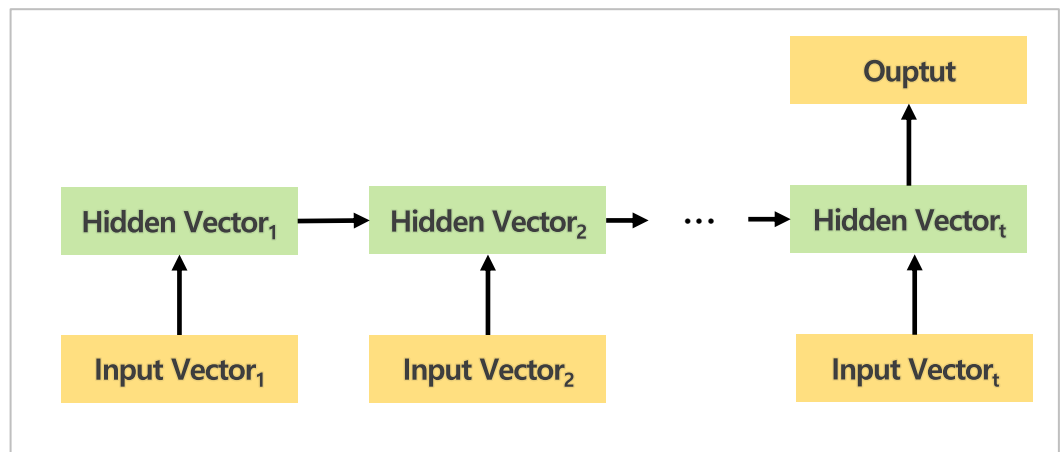
[1]



[2]



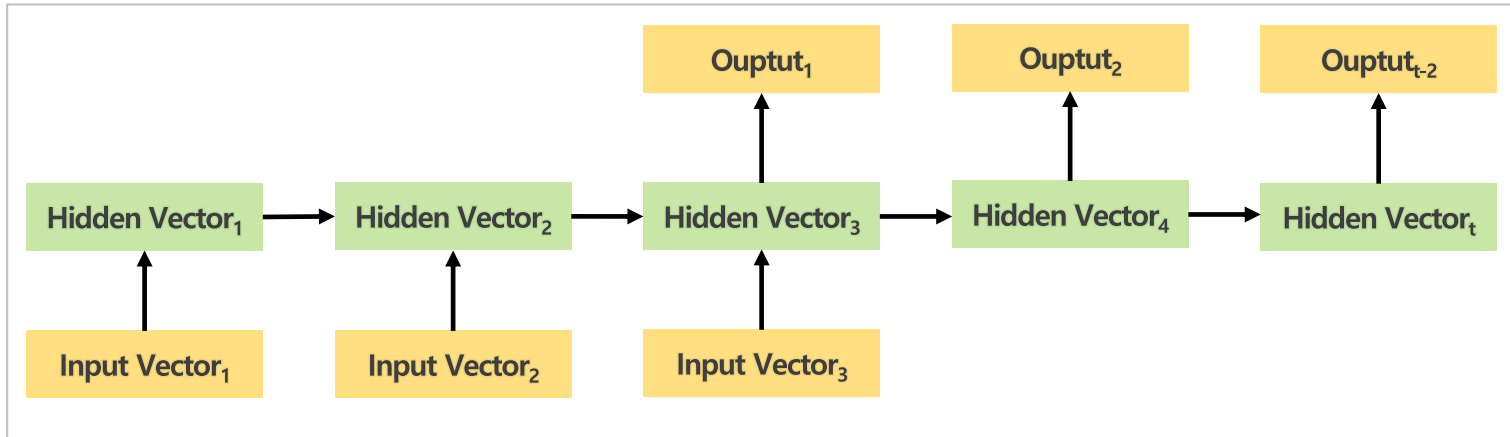
[3]



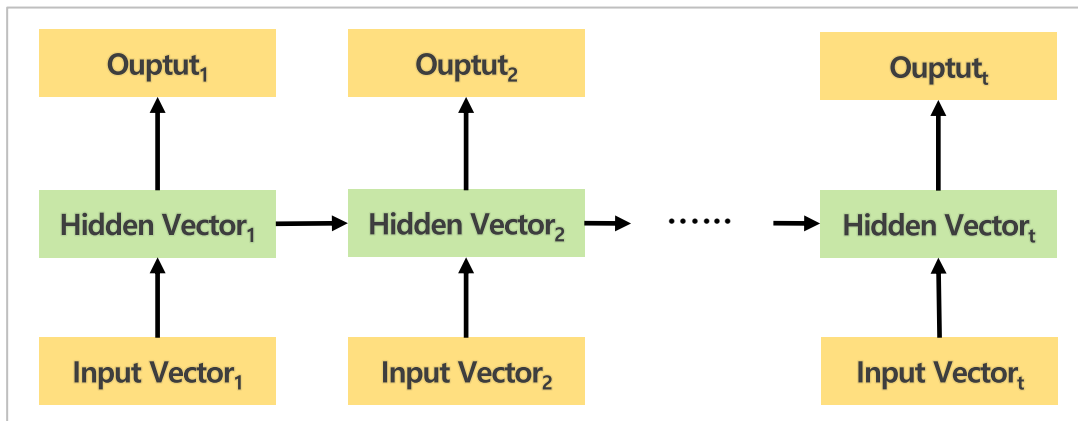
RNN (4)

- Typical RNN Structures (2)

[4]



[5]



Sentiment Analysis (1)

- **Sentiment Analysis**

- Called by 'Opinion mining' or 'emotion AI'
- The use of natural language processing, text analysis, computational linguistics, and biometrics to systematically identify, extract, quantify, and study affective states and subjective information
- Widely applied to voice of the customer materials such as reviews and survey responses, online and social media

(Ex)

Sentiment Analysis



Negative

I'm dissatisfied with your customer service.
No one was able to help me with the problems I had with using your product.



Neutral

The product has multiple features that are suitable for users with different levels of experience.

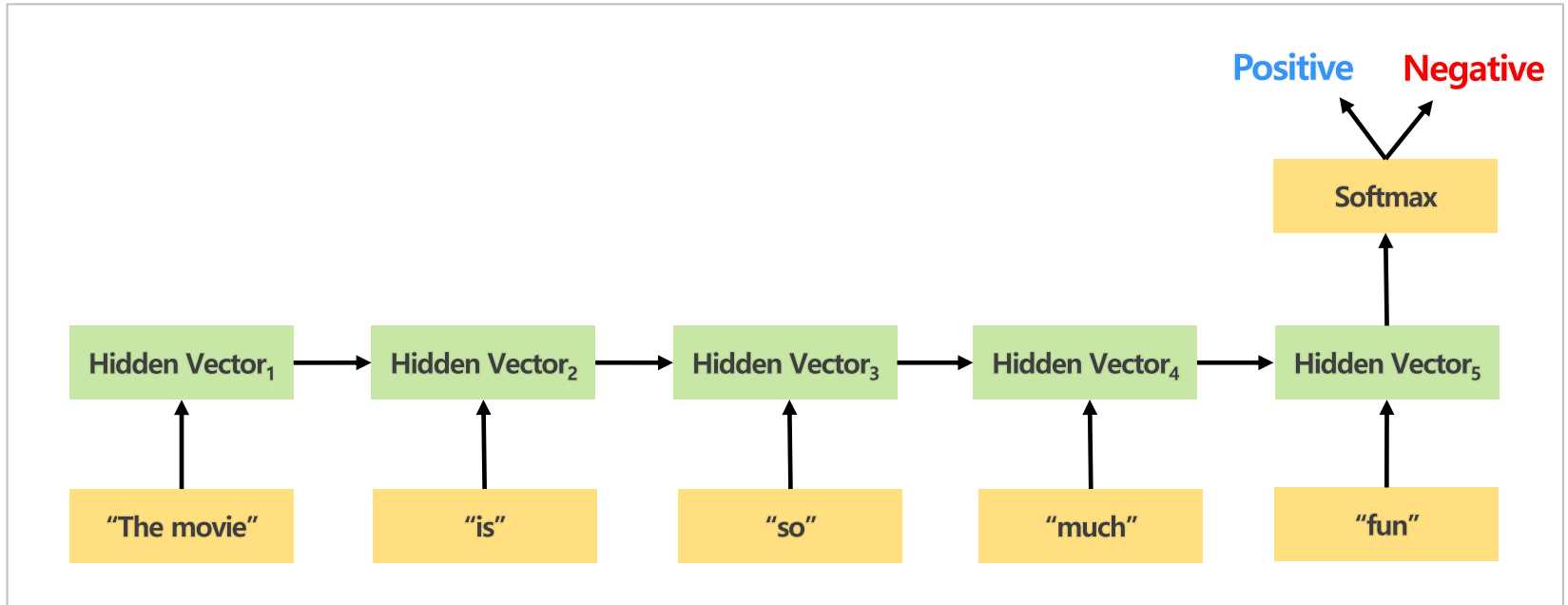


Positive

I really enjoy how easy this product is to use and how it successfully helps my team complete their day-to-day tasks.

Sentiment Analysis (2)

- Sentiment Analysis



(Example) Movie review

Sentiment Analysis (3) - Dataset

- IMDB
 - A dataset of movie reviews
 - <https://www.imdb.com/interfaces/>
 - Total 50,000 movie reviews
 - Each review consists of many English sentences
 - Label: 2 classes
 - Positive: 2
 - Negative: 1

One of the other reviewers has mentioned that after watching just 1 Oz episode you'll be hooked. The...	positive
A wonderful little production. The filming technique is very unassuming- very old-time-B...	positive
I thought this was a wonderful way to spend time on a too hot summer weekend, sitting in the air con...	positive
Basically there's a family where a little boy (Jake) thinks there's a zombie in his closet & his par...	negative
Petter Mattei's "Love in the Time of Money" is a visually stunning film to watch. Mr. Mattei offers ...	positive
Probably my all-time favorite movie, a story of selflessness, sacrifice and dedication to a noble ca...	positive
I sure would like to see a resurrection	positive

Sentiment Analysis (4) - Tokenizing

- Text dataset such as IMDB
 - A set of natural language texts, not a tensor or vector of numbers
 - To input such data into ANN, it is necessary to represent the data as numbers through a pre-processing process
- Tokenizing
 - Divide them into **tokens**, which are the smallest units of language
 - How to tokenize
 - Use split() function in python (very simple case)
 - Use Spacy library

Sentiment Analysis (5) - Tokenizing

- Tokening examples

[1]

“quick brown fox jumps over the lazy dog.”



[“quick”, “brown”, “fox”, “jumps”, “over”, “the”, “lazy”, “dog.”]

[2]

“한글 자연어 처리는 재밋다.”



[“한글”, “자연어”, “처리는”, “재밋다”, “.”]

Sentiment Analysis (6) - Tokenizing

- Tokenizing Libraries

- NTLK (Natural Language Toolkit)

- (Install) Conda install nltk or pip install nltk

```
from nltk.tokenize import word_tokenize
```

```
sentence = "Natural language processing (NLP) is a subfield of computer science, information engineering, and artificial intelligence"
```

```
print(word_tokenize(sentence))
```

```
>> Output
```

```
['Natural', 'language', 'processing', '(', 'NLP', ')', 'is', 'a', 'subfield', 'of', 'computer', 'science', ',', 'information', 'engineering', 'and', 'artificial', 'intelligence']
```

- Spacy

- (install) Conda install spacy or pip install spacy

```
nlp = spacy.load('en')
```

```
sentence = "Natural language processing (NLP) is a subfield of computer science, information engineering, and artificial intelligence"
```

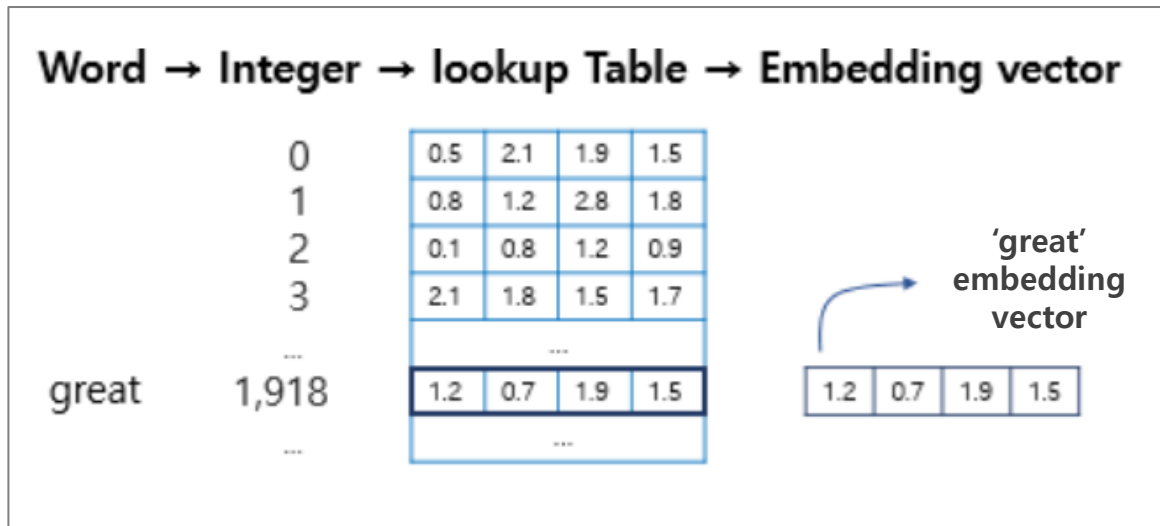
```
doc = nlp(sentence)
```

```
word_tokenized_sentence = [token.text for token in doc]
```

```
print(word_tokenized_sentence)
```

Sentiment Analysis (7) - Embedding

- Word Embedding
 - Represent all tokens as vectors
 - Need to define a dictionary containing as many vectors as there are all tokens
 - How to embed



Sentiment Analysis (8) - Embedding

- Implementation example

```
train_data = 'you need to know how to code'

word_set = set(train_data.split())
vocab = {word: i+2 for i, word in enumerate(word_set)}
vocab['<unk>'] = 0
vocab['<pad>'] = 1
print(vocab)
```

➡ {'code': 2, 'you': 3, 'know': 4, 'to': 5, 'need': 6, 'how': 7, '<unk>': 0, '<pad>': 1}

```
embedding_table = torch.FloatTensor([ [ 0.0, 0.0, 0.0], [ 0.0, 0.0, 0.0],
                                       [ 0.2, 0.9, 0.3], [ 0.1, 0.5, 0.7],
                                       [ 0.2, 0.1, 0.8], [ 0.4, 0.1, 0.1],
                                       [ 0.1, 0.8, 0.9], [ 0.6, 0.1, 0.1]])
```

```
sample = 'you need to run'.split() idxes = []
for word in sample:
    try: idxes.append(vocab[word])
    except KeyError:
        idxes.append(vocab['<unk>'])

idxes = torch.LongTensor(idxes)
lookup_result = embedding_table[idxes, :]
print(lookup_result)
```



```
tensor([[0.1000, 0.5000, 0.7000],
        [0.1000, 0.8000, 0.9000],
        [0.4000, 0.1000, 0.1000],
        [0.0000, 0.0000, 0.0000]])
```

Sentiment Analysis (Movie review) in Pytorch (1)

1. NL (Natural Language) Pre-processing

```
import os
import torch
import torch.nn as nn
import torch.nn.functional as F
from torchtext import data, datasets
```

2. Set hyper parameters

```
BATCH_SIZE = 64
lr = 0.001
EPOCHS = 10
USE_CUDA = torch.cuda.is_available()
DEVICE = torch.device("cuda" if USE_CUDA else "cpu")
```

3. Load dataset, Tensorize them, and Make a dictionary

```
TEXT = data.Field(sequential=True, batch_first=True, lower=True)
LABEL = data.Field(sequential=False, batch_first=True)

trainset, testset = datasets.IMDB.splits(TEXT, LABEL)

TEXT.build_vocab(trainset, min_freq=5)
LABEL.build_vocab(trainset)
```

Sentiment Analysis (Movie review) in Pytorch (2)

4. Split the original trainset into trainset and validationset

```
trainset, valset = trainset.split(split_ratio=0.8)
train_iter, val_iter, test_iter = data.BucketIterator.splits(
    (trainset, valset, testset), batch_size=BATCH_SIZE,
    shuffle=True, repeat=False)
```

```
vocab_size = len(TEXT.vocab)
n_classes = 2
```

```
print("[학습셋]: %d [검증셋]: %d [테스트셋]: %d [단어수]: %d [클래스] %d"
      % (len(trainset), len(valset), len(testset), vocab_size, n_classes))
```

➡ [학습셋]: 20000 [검증셋]: 5000 [테스트셋]: 25000 [단어수]: 46159 [클래스] 2

5. Define RNN model (GRU)

```
class BasicGRU(nn.Module):
    def __init__(self, n_layers, hidden_dim, n_vocab, embed_dim, n_classes, dropout_p=0.2):
        super(BasicGRU, self).__init__()
        print("Building Basic GRU model...")
        self.n_layers = n_layers

        self.embed = nn.Embedding(n_vocab, embed_dim)
        self.hidden_dim = hidden_dim
        self.dropout = nn.Dropout(dropout_p)
        self.gru = nn.GRU(embed_dim, self.hidden_dim, num_layers=self.n_layers, batch_first=True)
        self.out = nn.Linear(self.hidden_dim, n_classes)
```

Sentiment Analysis (Movie review) in Pytorch (3)

5-1. Define RNN model (GRU) (2)

```
def forward(self, x):  
    x = self.embed(x)  
    h_0 = self._init_state(batch_size=x.size(0))  
    x, _ = self.gru(x, h_0) # [i, b, h]  
    h_t = x[:, -1, :]  
    self.dropout(h_t)  
    logit = self.out(h_t) # [b, h] -> [b, o]  
    return logit  
  
def _init_state(self, batch_size=1):  
    weight = next(self.parameters()).data  
    return weight.new(self.n_layers, batch_size, self.hidden_dim).zero_()
```

Dictionary

quick: [0.1 0.2]
brown: [0.7 0.5]
fox: [0.3 0.3]
jumps: [0.9 0.1]
over: [0.01 0.8]
the: [0.3 0.6]
lazy: [0.2 0.7]
dog: [0.8 0.9]

Embedding

[quick brown fox jumps over the lazy dog]



[[0.1 0.2] [0.7 0.5] [0.3 0.3] [0.9 0.1] [0.01 0.8] [0.3 0.6] [0.2 0.7] [0.8 0.9]]

Sentiment Analysis (Movie review) in Pytorch (4)

6. Define train module

```
def train(model, optimizer, train_iter):
    model.train()
    for b, batch in enumerate(train_iter):
        x, y = batch.text.to(DEVICE), batch.label.to(DEVICE)
        y.data.sub_(1) # 레이블 값을 0과 1로 변환
        optimizer.zero_grad()

        logit = model(x)
        loss = F.cross_entropy(logit, y)
        loss.backward()
        optimizer.step()
```

7. Define evaluation module

```
def evaluate(model, val_iter):
    model.eval()
    corrects, total_loss = 0, 0
    for batch in val_iter:
        x, y = batch.text.to(DEVICE), batch.label.to(DEVICE)
        y.data.sub_(1) # 레이블 값을 0과 1로 변환
        logit = model(x)
        loss = F.cross_entropy(logit, y, reduction='sum')
        total_loss += loss.item()
        corrects += (logit.max(1)[1].view(y.size()).data == y.data).sum()
    size = len(val_iter.dataset)
    avg_loss = total_loss / size
    avg_accuracy = 100.0 * corrects / size
    return avg_loss, avg_accuracy
```

Sentiment Analysis (Movie review) in Pytorch (5)

8. Define GRU model

```
model = BasicGRU(1, 256, vocab_size, 128, n_classes, 0.5).to(DEVICE)
optimizer = torch.optim.Adam(model.parameters(), lr=lr)
```

9. Run

```
best_val_loss = None
for e in range(1, EPOCHS+1):
    train(model, optimizer, train_iter)
    val_loss, val_accuracy = evaluate(model, val_iter)

    print("[이폭: %d] 검증 오차:%5.2f | 검증 정확도:%5.2f" % (e, val_loss, val_accuracy))

    if not best_val_loss or val_loss < best_val_loss:
        if not os.path.isdir("snapshot"):
            os.makedirs("snapshot")
        torch.save(model.state_dict(), './snapshot/txtclassification.pt')
        best_val_loss = val_loss
```



```
[이폭: 1] 검증 오차: 0.69 | 검증 정확도:49.00
[이폭: 2] 검증 오차: 0.69 | 검증 정확도:50.00
[이폭: 3] 검증 오차: 0.69 | 검증 정확도:54.00
      ~~~
[이폭: 8] 검증 오차: 0.36 | 검증 정확도:86.00
[이폭: 9] 검증 오차: 0.38 | 검증 정확도:86.00
[이폭: 10] 검증 오차: 0.41 | 검증 정확도:86.00
```

Q & A
